

ESTUDIO — Carpeta entities/

Material de estudio para el final del TFC Bajoneá. Basado en el código real de `backend/src/main/java/com/bajonea/backend/entities/` — **27 archivos** (26 entidades + 1 clase de apoyo `@Embeddable`).

Qué es una Entity

Una **entity** es una clase Java que **representa una tabla de la base de datos**. Cada atributo de la clase es una columna, y cada objeto de esa clase es una fila.

Eso es un **ORM** (Object-Relational Mapping): la herramienta que traduce entre "el mundo de objetos" de Java y "el mundo de tablas" de SQL. En este proyecto el ORM es **Hibernate**, usado a través de **JPA** (que es el estándar; Hibernate es la implementación).

Lo importante: en Bajoneá **no se escribe SQL a mano casi nunca**. Vos escribís `productoRepository.save(producto)` y Hibernate genera el `INSERT INTO producto (...) VALUES (...)` solo.

Las anotaciones que se repiten en todas las entities

Anotación	Qué significa en criollo
<code>@Entity</code>	"Esta clase es una tabla."
<code>@Table(name = "producto")</code>	El nombre exacto de la tabla en MySQL.
<code>@Id</code>	Este campo es la clave primaria .
<code>@GeneratedValue(strategy = IDENTITY)</code>	El id lo genera MySQL solo (autoincremental).
<code>@Column(name = "...", length = N, nullable = false, unique = true)</code>	Cómo es la columna: nombre, largo, si admite nulos, si es única.
<code>@Enumerated(EnumType.STRING)</code>	Si el campo es un enum, guardarlo como texto ("ACTIVO") y no como número.
<code>@ManyToOne</code> , <code>@OneToMany</code> , <code>@OneToOne</code>	Las relaciones entre tablas.
<code>@JoinColumn(name = "comercio_id")</code>	La columna de clave foránea que materializa la relación.
<code>@MapsId</code>	"Mi id es el mismo que el de la entidad con la que me relaciono." Es la clave de la herencia del proyecto.
<code>@EmbeddedId</code> / <code>@Embeddable</code>	Para claves primarias compuestas (dos columnas).

Las convenciones propias del proyecto (reglas no negociables)

Estas están escritas en CLAUDE.md §4 y valen para toda entidad, sin excepción:

- `@Getter` a nivel de clase, pero `@Setter` campo por campo. Nunca `@Data` de Lombok — `@Data` genera `equals` / `hashCode` sobre todos los campos, y en relaciones bidireccionales eso provoca **recursión infinita** (A pregunta por B, B pregunta por A, y así hasta que revienta).
- El campo `id` **NUNCA** tiene `@Setter` . Ni siquiera en las entidades con id externo (`Provincia` , `Localidad`). El id es la identidad de la fila; si lo pudieras cambiar a mano, podrías pisar otra fila sin querer.
- `@EqualsAndHashCode(of = "id")` explícito. Dos objetos son "el mismo" si tienen el mismo id, sin importar el resto de los campos.
- `fetch = FetchType.LAZY` en toda relación. Concepto importante, ver abajo.
- Cero comentarios adentro de `entities/`** . Toda explicación de diseño vive en la documentación, nunca en el código de la entidad.

LAZY vs EAGER — pregunta muy probable en la mesa

	LAZY (lo que usa el proyecto)	EAGER
Cuándo trae la relación	Solo si la pedís	Siempre, junto con la entidad
Ejemplo	Traés un <code>Producto</code> , y su <code>Comercio</code> no se consulta hasta que hagas <code>producto.getComercio()</code>	Traés un <code>Producto</code> y automáticamente se trae el <code>Comercio</code> también

Por qué LAZY en todo: si fuera EAGER, traer un producto traería su comercio, y el comercio traería su dueño, y el dueño su persona jurídica... una sola consulta terminaría arrastrando media base. Con LAZY traés solo lo que necesitás.

El precio de LAZY es el famoso problema **N+1**: si recorrés 100 productos y para cada uno pedís su comercio, hacés 1 consulta + 100 consultas = 101 viajes a la base. Se resuelve con `JOIN FETCH` en las consultas que sí necesitan la relación.

GRUPO 1 — LA CADENA DE IDENTIDAD (8 entidades)

Esta es la **parte más conceptual del modelo** y la más probable de que te pregunten. Vale la pena entenderla bien.

El problema que resuelve

Bajoneá tiene tres tipos de usuario: **Cliente**, **Dueño** de comercio y **Administrador**. Los tres tienen email y contraseña. Pero:

- El Cliente y el Administrador son **personas físicas** (nombre, apellido, DNI, fecha de nacimiento).
- El Dueño representa una **persona jurídica** (razón social, CUIT, condición de IVA) y **además** tiene un representante que es una persona física.

Poner todos esos campos en una sola tabla `usuario` dejaría la mitad en `NULL` en cada fila. Feo y propenso a errores.

La solución: una cadena de tablas



Cómo se implementa: con `@MapsId`. Significa "mi clave primaria es la misma que la de mi padre". Si el `Usuario` tiene id 42, entonces su `Persona` tiene id 42, su `PersonaFisica` tiene id 42 y su `Cliente` tiene id 42. **Todas la misma fila conceptual, repartida en varias tablas.**

Esta técnica se llama **herencia por tabla unida** (*joined table inheritance*).

Ventajas: cada tabla tiene solo sus columnas, sin nulos innecesarios; la integridad la garantiza la base con FKs; y agregar un tipo de usuario nuevo es agregar una tabla, no modificar las existentes.

Desventaja (decila vos antes de que te la marquen): para armar un cliente completo hacen falta varios `JOIN`. Es el precio de la normalización.

1.1 Usuario — la identidad base

Qué representa: una cuenta del sistema. Es la raíz de toda la cadena.

Atributo	Tipo	Qué significa
id	Integer	Clave primaria, autogenerada por MySQL.
email	String(254)	El email de login. <code>unique = true</code> — no puede repetirse.
passwordHash	String(255)	La contraseña hasheada con BCrypt . Nunca en texto plano.
rol	RolUsuario (enum)	CLIENTE / DUENO / ADMINISTRADOR.
estado	EstadoUsuario (enum)	ACTIVO / PENDIENTE / BLOQUEADO / etc.
intentosFallidos	int	Cuántas veces erró la contraseña seguidas. A los 3, se bloquea.
fotoPerfilUrl	String(500)	URL de Cloudinary de su foto.
fechaRegistro	LocalDateTime	Cuándo se creó. <code>updatable = false</code> — no se puede cambiar nunca.
fechaUltimoAcceso	LocalDateTime	Último login.
fechaActualizacion	LocalDateTime	Última modificación.

Relaciones: ninguna declarada hacia abajo. Son los hijos (`Persona`) los que apuntan hacia arriba. Eso es a propósito: `Usuario` no necesita saber qué tipo de persona es.

Detalles para la mesa:

- Se llama `passwordHash` y no `password` — el nombre mismo documenta que **nunca** hay una contraseña en claro ahí adentro.
- `updatable = false` en `fechaRegistro` es una garantía a nivel ORM: aunque alguien intente cambiarla en código, Hibernate no la incluye en el `UPDATE`.

1.2 Persona — el nodo intermedio

Qué representa: el eslabón entre `usuario` y los dos tipos de persona. Es una tabla que casi no tiene datos propios — su función es **estructural**.

Atributo	Tipo	Qué significa
id	Integer	El mismo que el del Usuario. Sin <code>@GeneratedValue</code> .
usuario	Usuario	<code>@OneToOne</code> con <code>@MapsId</code> — su padre.

Por qué existe si no tiene datos: porque permite que `PersonaFisica` y `PersonaJuridica` cuelguen del mismo punto sin que `Usuario` tenga que conocerlas. Es un punto de extensión: si mañana hay un tercer tipo de persona, se agrega ahí sin tocar nada.

1.3 PersonaFisica — los datos de una persona de carne y hueso

Atributo	Tipo	Qué significa
id	Integer	El mismo que la Persona.
persona	Persona	<code>@OneToOne</code> + <code>@MapsId</code> .
nombre	<code>String(100)</code>	Obligatorio.
apellido	<code>String(100)</code>	Obligatorio.
dni	<code>String(10)</code>	Obligatorio y único .
fechaNacimiento	<code>LocalDate</code>	Obligatorio.
telefono	<code>String(30)</code>	Obligatorio.
fechaModificacion	<code>LocalDateTime</code>	

Por qué el DNI es `String` y no un número: porque un DNI **no es una cantidad**, es un identificador. Nunca vas a sumar dos DNIs. Además, si fuera número, un DNI que empiece con cero perdería el cero. Misma lógica que el CUIT y el código postal.

1.4 PersonaJuridica — los datos de una empresa

Atributo	Tipo	Qué significa
id	Integer	El mismo que la Persona.
persona	Persona	<code>@OneToOne</code> + <code>@MapsId</code> .
razonSocial	<code>String(150)</code>	El nombre legal de la empresa.
cuit	<code>String(11)</code>	Obligatorio y único . Exactamente 11 dígitos.
condicionIva	<code>CondicionIva</code> (enum)	Responsable inscripto, monotributista, etc.
tipoSociedad	<code>TipoPersonaJuridica</code> (enum)	SRL, SA, etc.
domicilioFiscal	<code>String(255)</code>	La dirección legal.
fechaInicioActividades	<code>LocalDate</code>	Cuándo empezó a facturar.

Diferencia razón social vs nombre del comercio: la razón social es el nombre legal ("Gastronómica del Sur S.R.L."), el nombre del comercio es el de fantasía ("Pizzas del Sur"). Por eso son campos distintos, en tablas distintas.

1.5 Cliente

Atributo	Tipo	Qué significa
id	Integer	El mismo que la PersonaFisica.
personaFisica	PersonaFisica	<code>@OneToOne</code> + <code>@MapsId</code> .

Es una tabla con una sola columna, y eso está bien. No agrega ningún dato propio — su función es **marcar el rol**. Existe la fila en `cliente` con id 42 → el usuario 42 es cliente.

Podría discutirse si hace falta, pero tiene una ventaja concreta: te permite poner FKs hacia `cliente` (el carrito, los pedidos, las direcciones apuntan ahí), y así la base **garantiza** que un pedido solo puede pertenecer a alguien que efectivamente es cliente.

1.6 Administrador

Atributo	Tipo	Qué significa
id	Integer	El mismo que la PersonaFisica.
personaFisica	PersonaFisica	<code>@OneToOne</code> + <code>@MapsId</code> .

Igual que `Cliente` . **No tiene registro público** — el administrador se siembra directamente en la base con una migración de Flyway. No hay pantalla de "regístrate como admin", por razones obvias.

1.7 Dueno — la entidad que más cambió en el proyecto

Qué representa: la persona (jurídica) que es dueña de uno o varios comercios.

Atributo	Tipo	Qué significa
id	Integer	El mismo que la <code>PersonaJuridica</code> .
personaJuridica	PersonaJuridica	<code>@OneToOne</code> + <code>@MapsId</code> — los datos de la empresa.
personaFisica	PersonaFisica	<code>@OneToOne</code> con FK propia <code>persona_fisica_id</code> , <code>nullable=false</code> , <code>unique=true</code> — el representante legal .
fechaCreacion	LocalDateTime	<code>updatable = false</code> .

Esto es lo más interesante del modelo, y es buena carne de mesa

El `Dueno` tiene DOS relaciones a la vez:

- `@MapsId` con `PersonaJuridica` — "*yo **soy** esta empresa*". Comparten id.
- `@OneToOne` normal con `PersonaFisica` — "*esta persona **me representa***". FK aparte.

Es la primera relación del proyecto que **no** usa `@MapsId` en una entidad de la cadena de identidad, y por eso lleva `unique = true` : una persona física no puede ser representante de dos dueños distintos.

El cambio de cardinalidad — el mayor cambio arquitectónico del proyecto

En el MVP original era así:

`Usuario 1 <-> 1 Comercio` (un usuario = un comercio)

Ahora es así:

`Usuario -> Dueno 1 <-> N Comercio` (un dueño puede tener varios comercios)

Por qué se cambió: en la realidad, alguien puede tener una pizzería y una hamburguesería. Con el modelo viejo tendría que crear dos cuentas separadas.

Qué implicó: este cambio tocó `ComercioService` , `ProductoService` , `PedidoService` y `AuthService` a la vez. Fue identificado desde el principio como **el riesgo técnico más grande** de la portabilidad al modelo completo. También es la razón de que el rol se llame `DUENO` y no `COMERCIO` : el rol le pertenece a la persona, no al negocio.

1.8 Comercio — el negocio

Atributo	Tipo	Qué significa
id	Integer	Clave primaria propia, autogenerada. No usa <code>@MapsId</code> .
dueno	Dueno	<code>@ManyToOne</code> , <code>nullable=false</code> — muchos comercios pertenecen a un dueño .
nombre	String(150)	El nombre de fantasía.
descripcion	String (TEXT)	Texto libre largo.
fotoPerfilUrl	String(500)	Su logo/foto.
telefono	String(30)	
email	String(150)	El email de contacto público (distinto del de login).
tipoComercio	TipoComercio (enum)	Pizzería, parrilla, etc.
aceptaDelivery	boolean	Si hace envíos.
aceptaRetiro	boolean	Si se puede retirar en el local.
estado	EstadoComercio (enum)	PENDIENTE / APROBADO / RECHAZADO.
fechaRegistro	LocalDateTime	<code>updatable = false</code> .
fechaModificacion	LocalDateTime	

En criollo: "*muchos comercios pertenecen a un dueño*". La FK `dueno_id` vive en la tabla `comercio` — la regla de las relaciones N:1 es que la FK va del lado del "muchos".

El `estado` es la clave del flujo de aprobación: un comercio nace `PENDIENTE` , no aparece en el catálogo público, y solo pasa a `APROBADO` cuando el administrador lo revisa.

GRUPO 2 — SEGURIDAD Y CUENTA (2 entidades)

2.1 Sesion — la que hace que el JWT se pueda cancelar

Qué representa: cada vez que alguien se loguea, se crea una fila acá.

Atributo	Tipo	Qué significa
id	Integer	Autogenerado. Este id va adentro del JWT.
usuario	Usuario	@ManyToOne — un usuario puede tener muchas sesiones a lo largo del tiempo.
activa	boolean	El campo clave. Si es false , el token asociado deja de servir.
fechaInicio	LocalDateTime	updatable = false .
fechaCierre	LocalDateTime	Cuándo se cerró (null si sigue abierta).
tipoCierre	TipoCierreSesion (enum)	Por qué se cerró: manual, bloqueo, cambio de contraseña...
ipOrigen	String(45)	Desde qué IP. 45 caracteres porque una IPv6 puede ser larga.
navegador	String(255)	El User-Agent.
dispositivo	String(255)	

Por qué es importante (repite lo de ESTUDIO-CONFIG.md pero desde el modelo): un JWT stateless no se puede revocar. Con esta tabla, JwtAuthenticationFilter chequea activa en cada request, y poner activa = false invalida el token al instante. Se usa en: logout, bloqueo de cuenta, cambio de contraseña, y login concurrente.

El tipoCierre es auditoría real: podés mirar la tabla y saber si un usuario cerró sesión él mismo o si el sistema se la cerró por seguridad.

2.2 Token — los códigos temporales

Qué representa: un código de un solo uso, con vencimiento. Se usa para verificar el email, recuperar la contraseña y reactivar la cuenta.

Atributo	Tipo	Qué significa
id	Integer	
usuario	Usuario	@ManyToOne — de quién es.
tipo	TipoToken (enum)	VERIFICACION_EMAIL / RECUPERACION_PASSWORD / REACTIVACION_CUENTA.
token	String(36)	Único. El valor propiamente dicho.
fechaCreacion	LocalDateTime	
fechaVencimiento	LocalDateTime	Después de esta fecha no sirve más.
estado	EstadoToken (enum)	Pendiente / usado / vencido...
fechaUso	LocalDateTime	Cuándo se consumió.
intentosFallidos	int	Cuántas veces se intentó con un código equivocado.

Detalles:

- length = 36 es exactamente el largo de un UUID (550e8400-e29b-41d4-a716-446655440000). Los códigos de 6 dígitos que se usan ahora entran holgados en ese campo.
- Vencimientos reales del proyecto: 30 minutos para recuperación de contraseña, 24 horas para reactivación de cuenta.
- intentosFallidos a nivel de token evita que alguien pruebe los 999.999 códigos posibles por fuerza bruta.

GRUPO 3 — CATÁLOGO DE PRODUCTOS (5 entidades)

3.1 Producto

Atributo	Tipo	Qué significa
id	Integer	
comercio	Comercio	@ManyToOne , nullable=false — muchos productos pertenecen a un comercio.
categoria	Categoria	@ManyToOne , nullable=false — muchos productos son de una categoría.
nombre	String(150)	
descripcion	String (TEXT)	
precio	BigDecimal(10,2)	Hasta 10 dígitos, 2 decimales.
estado	EstadoProducto (enum)	DISPONIBLE / AGOTADO / DESCONTINUADO.
fechaCreacion , fechaModificacion , fechaBaja	LocalDateTime	
imagenes	List<ImagenProducto>	@OneToMany(mappedBy = "producto") — un producto tiene muchas imágenes.

El mappedBy = "producto" explicado: significa "esta relación ya está mapeada del otro lado, en el campo producto de ImagenProducto ". O sea: la FK vive en la tabla imagen_producto , no acá. Sin mappedBy , Hibernate crearía una tercera tabla intermedia innecesaria.

Fijate que Producto no tiene una columna de imagen. Todas las imágenes están en la otra tabla. Eso es lo correcto: si tuviera imagen1 , imagen2 ... imagen5 como columnas, estarías violando la primera forma normal.

3.2 Categoria y 3.3 Tag

Ambas con la misma estructura exacta:

Atributo	Tipo	Qué significa
id	Integer	
nombre	String(100)	Único.
activo	boolean	Para la baja lógica.
fechaCreacion , fechaModificacion , fechaBaja	LocalDateTime	

Baja lógica, otra vez: el DELETE de la API pone activo = false y llena fechaBaja . La fila sigue existiendo. Así los productos que la usaban no quedan huérfanos, y se puede reactivar.

Ninguna de las dos tiene una lista de productos. La relación se navega desde el producto hacia la categoría, no al revés. Es a propósito: si Categoria tuviera List<Producto> , cargar una categoría con 500 productos sería carísimo, y en la práctica nunca hace falta.

3.4 ProductoTag + ProductoTagId — la única clave compuesta del proyecto

Qué representa: la tabla intermedia de la relación muchos a muchos entre productos y tags.

ProductoTagId (@Embeddable)

Atributo	Tipo
productoId	Integer
tagId	Integer

No es una entidad, es un @Embeddable : no tiene tabla propia, es "un pedazo" que se incrusta adentro de otra entidad. Implementa Serializable porque JPA lo exige para las claves compuestas.

ProductoTag (la entidad)

Atributo	Tipo	Qué significa
id	ProductoTagId	@EmbeddedId — la clave primaria es el par (productId, tagId).
producto	Producto	@ManyToOne + @MapsId("productId") .
tag	Tag	@ManyToOne + @MapsId("tagId") .

Por qué esto es así — explicación para la mesa

Una relación **N:M** no se puede representar con una FK, porque una columna guarda un solo valor. Un producto tiene varios tags y un tag está en varios productos. La solución universal en bases relacionales es una **tabla intermedia** donde cada fila es un par:

producto_id	tag_id
5	2
5	7
8	2

La clave primaria es **el par completo**, porque ningún producto puede tener el mismo tag dos veces.

Detalle sobre las reglas del proyecto: la regla dice "el campo `id` nunca lleva `@Setter`". Acá `ProductoTagId` **sí** tiene `@Setter` en sus dos campos. No es una violación: sus campos se llaman `productoId` y `tagId`, no `id`. Y `ProductoTag` respeta la regla — su `@EmbeddedId id` no tiene setter.

3.5 ImagenProducto

Atributo	Tipo	Qué significa
<code>id</code>	<code>Integer</code>	
<code>producto</code>	<code>Producto</code>	<code>@ManyToOne</code> , <code>nullable=false</code> — muchas imágenes de un producto.
<code>url</code>	<code>String(500)</code>	La URL en Cloudinary.
<code>orden</code>	<code>Integer</code>	En qué posición se muestra.
<code>esPrincipal</code>	<code>boolean</code>	Cuál es la foto de portada.

Máximo **5 imágenes por producto**. Ese límite **no está en la entidad** — se valida en `ProductoService`, que devuelve 409 al intentar la sexta. Es una regla de negocio, y las reglas de negocio van en el service.

GRUPO 4 — CARRITO (2 entidades)

4.1 Carrito

Atributo	Tipo	Qué significa
<code>id</code>	<code>Integer</code>	
<code>cliente</code>	<code>Cliente</code>	<code>@OneToOne</code> , <code>nullable=false</code> , <code>unique=true</code> — un carrito por cliente.
<code>comercio</code>	<code>Comercio</code>	<code>@ManyToOne</code> , puede ser null — a qué comercio está atado.

Las dos reglas de negocio que se leen del modelo

- Un solo carrito por cliente, siempre.** El `unique = true` en `cliente_id` lo garantiza a nivel base de datos. No hay carritos históricos ni múltiples.
- Un carrito solo puede tener productos de UN comercio a la vez.** Por eso el carrito tiene un `comercio` propio, y **puede ser null** cuando está vacío. Cuando agregás el primer producto, el carrito se "ata" a ese comercio; si intentás agregar uno de otro, el service devuelve **409 Conflict**. Al vaciarlo, el `comercio` vuelve a null y podés empezar en otro lado.

Por qué esa regla existe: cada comercio prepara y entrega por su cuenta. Un pedido con productos de dos comercios distintos sería inentregable.

4.2 ItemCarrito

Atributo	Tipo	Qué significa
<code>id</code>	<code>Integer</code>	
<code>carrito</code>	<code>Carrito</code>	<code>@ManyToOne</code> , <code>nullable=false</code> .
<code>producto</code>	<code>Producto</code>	<code>@ManyToOne</code> , <code>nullable=false</code> .
<code>cantidad</code>	<code>Integer</code>	Cuántos.
<code>nota</code>	<code>String(255)</code>	"sin cebolla", etc.

Ojo con esto, es un contraste importante: `ItemCarrito` **NO guarda el precio**. Lo lee del producto en el momento. Así, si el comercio cambia el precio mientras vos tenés algo en el carrito, ves el precio actualizado.

`DetallePedido` (el equivalente en un pedido) **SÍ guarda el precio**. La diferencia es clave y está explicada abajo.

GRUPO 5 — PEDIDO (2 entidades)

5.1 Pedido — la entidad más grande del proyecto (21 atributos)

Atributo	Tipo	Qué significa
id	Integer	El número de pedido.
cliente	Cliente	@ManyToOne , nullable=false — quién lo hizo.
comercio	Comercio	@ManyToOne , nullable=false — a quién.
direccion	Direccion	@ManyToOne , puede ser null (si es retiro en local).
tipoEntrega	TipoEntrega (enum)	Columna modalidad_entrega . DOMICILIO o RETIRO.
estado	EstadoPedido (enum)	En qué anda el pedido.
pagoEstado	EstadoPagoPedido (enum)	Estado del pago.
canceladoPor	CanceladoPor (enum)	Si se canceló, quién lo hizo.
motivo	String(500)	Motivo de cancelación.
fuelleEntrega	FuelleEntrega (enum)	Quién entrega.
fechaEntrega	LocalDateTime	Cuándo se entregó.
suspensionRetiroExpira	LocalDateTime	Hasta cuándo se guarda el pedido para retirar.
primerAvisoEmitido	boolean	Si ya se avisó al cliente.
motivoRechazo	MotivoRechazo (enum)	Si el comercio lo rechazó, por qué.
comentarioRechazo	String(500)	El detalle libre del rechazo.
subtotal	BigDecimal(10,2)	La suma de los productos.
cargoServicioCliente	BigDecimal(10,2)	Lo que le cobra la plataforma al cliente.
cargoServicioComercio	BigDecimal(10,2)	Lo que le cobra la plataforma al comercio.
total	BigDecimal(10,2)	El total final.
fechaCreacion	LocalDateTime	updatable = false .

Cosas para saber contar

- Los tres importes separados son el modelo de negocio de la plataforma. La plataforma cobra comisión de los dos lados. total = subtotal + cargoServicioCliente , y el comercio recibe subtotal - cargoServicioComercio . Tener las tres columnas guardadas (en vez de recalcularlas) significa que si mañana cambia la comisión, los pedidos viejos siguen mostrando lo que efectivamente se cobró.
- Hay campos que todavía no se usan del todo. pagoEstado , canceladoPor , fueuelleEntrega , suspensionRetiroExpira y primerAvisoEmitido pertenecen a la máquina de estados completa del proyecto (con MercadoPago), que está en el modelo de datos pero cuya lógica todavía no está implementada en los services. (Está documentado como tramo pendiente en CLAUDE.md §1bis — no lo presentes como funcionalidad terminada.)
- direccion es nullable a propósito: si el pedido es para retirar en el local, no hay dirección de entrega.
- La máquina de estados implementada hoy es acotada: PENDIENTE → EN_PREPARACION O PENDIENTE → RECHAZADO . El enum EstadoPedido tiene más valores (del modelo completo), pero los services solo manejan esas transiciones. (Ver ESTUDIO-ENUMS.md.)

5.2 DetallePedido — la línea del pedido

Atributo	Tipo	Qué significa
id	Integer	
pedido	Pedido	@ManyToOne , nullable=false .
producto	Producto	@ManyToOne , nullable=false .
cantidad	Integer	
precioUnitario	BigDecimal(10,2)	updatable = false — el precio congelado.
nota	String(255)	
subtotal	BigDecimal(10,2)	cantidad × precioUnitario.
estado	EstadoDetallePedido (enum)	Si esa línea puntual fue anulada.
motivoAnulacion	String(255)	Por qué se anuló.

El "snapshot de precio" — buena respuesta para la mesa

`precioUnitario` con `updatable = false` es el detalle más importante de esta entidad.

Cuando confirmás un pedido, el precio del producto se **copia** al detalle y **queda congelado para siempre**. Si mañana el comercio sube la pizza de \$8000 a \$9500, tu pedido de ayer sigue diciendo \$8000.

Por qué es imprescindible: un pedido es un **documento histórico** — es el comprobante de una transacción real. Si el detalle apuntara al precio actual del producto, tu historial de compras cambiaría solo cada vez que el comercio ajusta precios, y sería imposible auditar cuánto pagaste.

El `updatable = false` es la garantía técnica de eso: aunque alguien intente cambiarlo por código, Hibernate no lo incluye en el `UPDATE` .

Comparalo con `ItemCarrito` , que NO guarda el precio: el carrito es algo **vivo** (querés ver el precio de hoy), el pedido es algo **cerrado** (querés ver el precio de cuando compraste). Misma información, tratamiento opuesto, por una razón de negocio.

GRUPO 6 — SOPORTE Y ANEXOS (6 entidades)

6.1 Direccion

Atributo	Tipo	Qué significa
<code>id</code>	<code>Integer</code>	
<code>calle</code>	<code>String(150)</code>	
<code>numero</code>	<code>String(10)</code>	
<code>pisoDepto</code>	<code>String(30)</code>	Opcional.
<code>codigoPostal</code>	<code>String(10)</code>	
<code>localidad</code>	<code>Localidad</code>	<code>@ManyToOne</code> , <code>nullable=false</code> .
<code>cliente</code>	<code>Cliente</code>	<code>@ManyToOne</code> , nullable — muchas direcciones de un cliente.
<code>comercio</code>	<code>Comercio</code>	<code>@OneToOne</code> , <code>unique=true</code> , nullable — una dirección por comercio.
<code>principal</code>	<code>boolean</code>	Si es la dirección por defecto.
<code>eliminada</code>	<code>boolean</code>	Baja lógica.
<code>fechaCreacion</code> , <code>fechaModificacion</code> , <code>fechaBaja</code>	<code>LocalDateTime</code>	

La "exclusión mutua" — concepto que puede caer

Una dirección pertenece **o a un cliente, o a un comercio, nunca a los dos**. Ambos campos son nullable, y la regla es que **exactamente uno** tiene que estar lleno.

Esto la base no lo puede garantizar sola. Hay una anotación custom `@DireccionExclusionMutua` (ver ESTUDIO-VALIDATION.md), pero en la práctica la garantía viene del diseño: **no existe un endpoint genérico de "crear dirección"**. Las direcciones se crean solo dentro del registro de cliente o del registro de comercio, y cada flujo la asocia a lo suyo. Es imposible mandar las dos.

Nota sobre cardinalidad, que es sutil: un `Cliente` puede tener **muchas** direcciones (`@ManyToOne`), pero un `Comercio` solo **una** (`@OneToOne` con `unique`). Tiene lógica: vos podés pedir a casa o al trabajo, pero el local del comercio es uno solo.

6.2 Horario

Atributo	Tipo	Qué significa
<code>id</code>	<code>Integer</code>	
<code>comercio</code>	<code>Comercio</code>	<code>@ManyToOne</code> , <code>nullable=false</code> — muchos horarios de un comercio.
<code>diaSemana</code>	<code>DiaSemana</code> (enum)	Lunes a domingo.
<code>horaApertura</code>	<code>LocalTime</code>	
<code>horaCierre</code>	<code>LocalTime</code>	

Cómo se modela un horario, que es más elegante de lo que parece:

- **0 o más filas por día.**
- **Un día sin ninguna fila = cerrado ese día.** No hace falta un booleano `abre` .
- **Horario partido = dos filas del mismo día.** Ej: lunes 12:00-15:00 y lunes 20:00-00:00.

Es un modelo simple que cubre todos los casos sin campos extra.

Lo que valida el service, no la entidad: que `horaCierre` sea posterior a `horaApertura` . Es una regla entre dos campos, así que no puede ir como anotación de Bean Validation.

6.3 RedSocial

Atributo	Tipo	Qué significa
id	Integer	
comercio	Comercio	@ManyToOne , nullable=false .
tipo	TipoRedSocial (enum)	Instagram, Facebook, WhatsApp...
url	String(500)	El link.
fechaCreacion , fechaModificacion , fechaBaja	LocalDateTime	

Baja lógica, igual que categorías y tags. Máximo **5 por comercio**, mínimo 1 — reglas de negocio validadas en el DTO y en el service.

6.4 Notificacion

Atributo	Tipo	Qué significa
id	Integer	
usuario	Usuario	@ManyToOne , nullable=false — para quién es.
tipo	TipoNotificacion (enum)	Qué clase de aviso.
mensaje	String(500)	El texto.
leida	boolean	Si ya la vio.
fechaCreacion	LocalDateTime	
canal	CanalNotificacion (enum)	Por dónde va (in-app, email...).
estado	EstadoEnvioNotificacion (enum)	Si se envió bien.
fechaEnvio	LocalDateTime	
entidadTipo	TipoEntidadNotificacion (enum)	A qué se refiere.
entidadId	Integer	El id de esa cosa.

La referencia polimórfica — muy buen tema para la mesa

Antes, esta tabla tenía un campo `pedido_id` . Servía solo para notificaciones de pedido.

Ahora tiene el par `entidadTipo` + `entidadId` , que puede apuntar a cualquier cosa:

entidadTipo	entidadId	Significa
PEDIDO	42	Es sobre el pedido 42.
COMERCIO	7	Es sobre el comercio 7.

Ventaja: una sola tabla sirve para notificaciones de cualquier entidad, presente o futura. El frontend lee el par y sabe a qué pantalla mandarte al hacer clic.

Desventaja, y hay que decirla: la base no puede poner una FK — no sabe a qué tabla apunta `entidadId` . Si borras el pedido 42, la notificación queda apuntando a la nada, y MySQL no te avisa. La integridad queda a cargo de la aplicación. Es el trade-off clásico de este patrón: flexibilidad a cambio de garantías de la base.

6.5 HistorialEstadoComercio

Atributo	Tipo	Qué significa
id	Integer	
comercio	Comercio	@ManyToOne , nullable=false .
administrador	Administrador	@ManyToOne , nullable — quién hizo el cambio.
estadoOrigen	EstadoComercio	De qué estado venía (null si es el primero).
estadoDestino	EstadoComercio	A qué estado fue. nullable=false .
motivo	String(500)	Por qué.
fechaHora	LocalDateTime	Cuándo.

Es una tabla de auditoría "append-only": solo se insertan filas, nunca se modifican ni se borran. Cada aprobación o rechazo deja un registro permanente.

Por qué existe, técnicamente: el modelo de datos eliminó la columna `Comercio.motivo_rechazo` a favor de esta tabla. O sea, esta tabla es la única fuente del motivo de rechazo. Si querés saber por qué rechazaron un comercio, buscás acá su última transición a `RECHAZADO`.

El `administrador` es nullable porque puede haber transiciones automáticas del sistema, sin una persona detrás.

6.6 Provincia y Localidad — el catálogo geográfico

Provincia

Atributo	Tipo	Qué significa
id	String(2)	El código oficial. Sin <code>@GeneratedValue</code> .
nombre	String(100)	

Localidad

Atributo	Tipo	Qué significa
id	String(15)	El código oficial. Sin autogenerar.
nombre	String(150)	
provincia	Provincia	<code>@ManyToOne</code> , <code>nullable=false</code> — muchas localidades de una provincia.

Lo particular de estas dos

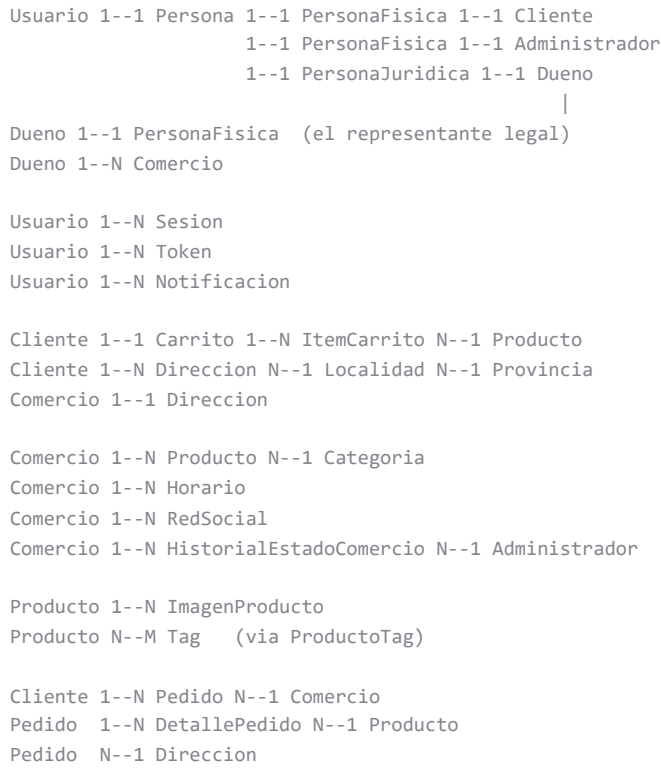
Son las únicas entidades sin `@GeneratedValue`. El id no lo genera MySQL — viene de afuera, de la API Georef del gobierno argentino, y son `String` porque son códigos oficiales, no números correlativos.

Cómo se cargaron: con un ETL (Extract-Transform-Load), un script Node.js (`et1-georef.mjs`) que consultó la API pública, transformó los datos y los insertó. Cargó **24 provincias y ~4038 localidades**.

Detalle que muestra buena práctica: el script es **idempotente** — usa `ON DUPLICATE KEY UPDATE`, así que lo podés correr dos veces y obtenés exactamente los mismos conteos, sin duplicados. Se verificó corriéndolo dos veces.

Anécdota real que quedó documentada: durante la carga hubo un problema de **codificación UTF-8** con los acentos (Río Grande, Tierra del Fuego). Se corrigió y se verificó **byte a byte** que los caracteres quedaran bien guardados.

Mapa completo de relaciones



Preguntas típicas de mesa

"¿Cómo modelaste los distintos tipos de usuario?" Con herencia por tabla unida (`@MapsId`). `Usuario` tiene lo común (email, password, rol); de ahí cuelga `Persona`, que se especializa en `PersonaFisica` o `PersonaJuridica`, y de esas cuelgan `Cliente`, `Administrador` y `Dueno`. Todos comparten el mismo id.

"¿Por qué el precio del pedido está duplicado si ya está en producto?" No está duplicado, está congelado. Un pedido es un documento histórico: tiene que mostrar el precio del momento de la compra, no el actual. Por eso `DetallePedido.precioUnitario` tiene `updatable = false`.

"¿Cómo hacés una relación muchos a muchos?" Con una tabla intermedia. En mi caso `ProductoTag`, con clave primaria compuesta por (`producto_id`, `tag_id`) usando `@EmbeddedId`.

5/9/26, 6:03 p.m.

Markdown Live Preview

"¿Qué es LAZY?" Que la relación no se trae de la base hasta que la pidas explícitamente. Uso LAZY en todas las relaciones para no arrastrar media base en cada consulta.

"¿Por qué no borrás las categorías de verdad?" Porque tendría productos apuntando a una categoría inexistente. Uso baja lógica: pongo `activo = false` . Los datos históricos quedan coherentes y se puede reactivar.

"¿Por qué el DNI es String?" Porque es un identificador, no una cantidad. Nunca lo vas a sumar, y si fuera número perderías los ceros a la izquierda.

"¿Cómo garantizás que un carrito tenga productos de un solo comercio?" El carrito tiene su propio campo `comercio` . Al agregar el primer producto se ata a ese comercio; si intentás uno de otro, el service devuelve 409.

Índice de entities cubiertas en este documento

Entity	Tabla	Qué representa
Usuario	usuario	La cuenta: email, contraseña, rol, estado.
Persona	persona	Nodo intermedio de la jerarquía.
PersonaFisica	persona_fisica	Nombre, apellido, DNI, fecha de nacimiento.
PersonaJuridica	persona_juridica	Razón social, CUIT, datos fiscales.
Cliente	cliente	El rol de comprador.
Administrador	administrador	El rol de gestión interna.
Dueno	dueno	El dueño de comercios, con su representante legal.
Comercio	comercio	El negocio gastronómico.
Sesion	sesion	Cada login; permite invalidar tokens.
Token	token	Códigos temporales de un solo uso.
Producto	producto	Un ítem del menú de un comercio.
Categoria	categoria	Clasificación de productos (una por producto).
Tag	tag	Etiqueta libre (varias por producto).
ProductoTag	producto_tag	Tabla intermedia N:M producto-tag.
ProductoTagId	(sin tabla)	@Embeddable con la clave compuesta.
ImagenProducto	imagen_producto	Una foto de la galería (máx. 5 por producto).
Carrito	carrito	Un carrito por cliente, atado a un comercio.
ItemCarrito	item_carrito	Una línea del carrito, sin precio guardado.
Pedido	pedido	El pedido confirmado, con sus importes.
DetallePedido	detalle_pedido	Una línea del pedido, con precio congelado.
Direccion	direccion	Dirección de un cliente o de un comercio.
Horario	horario	Una franja de atención del comercio.
RedSocial	red_social	Una red social del comercio.
Notificacion	notificacion	Aviso in-app con referencia polimórfica.
HistorialEstadoComercio	historial_estado_comercio	Auditoría de aprobaciones y rechazos.
Provincia	provincia	Catálogo geográfico, cargado por ETL.
Localidad	localidad	Catálogo geográfico, cargado por ETL.